

lab9_2414308_郭威威_实验9

数据库实体关系设计与实现实验报告

实验目的

1. 掌握实体关系图(ER图)的设计方法与规范
2. 理解1:1和1:N关系的数据库实现技术
3. 学习弱实体在关系数据库中的表示方法
4. 掌握通过外键约束实现实体间关系的技术

实验环境

- 数据库管理系统：MySQL 8.0
- 建模工具：ER图设计工具或绘图软件
- 数据库客户端：MySQL Workbench或命令行工具

ER图设计与分析

1. 实体识别与属性定义

1.1 主要实体

- **顾客(Customer)**
 - 主键：id_card (身份证号, VARCHAR(18))
 - 属性：name (姓名, VARCHAR(50))
- **会员卡(MembershipCard)**
 - 主键：card_number (卡号, VARCHAR(20))
 - 属性：level (等级, VARCHAR(10)), points (积分, INT)

1.2 弱实体

- **地址(Address)**
 - 主键：address_id (自增ID, INT)
 - 属性：province (省), city (市), district (区), street (街道)
- **电话(Phone)**

- 主键: phone_id (自增ID, INT)
- 属性: area_code (区号), number (号码)

2. 关系分析

2.1 顾客-会员卡关系

- 关系类型: 1:1
- 实现方式: 在MembershipCard表中添加UNIQUE外键
- 业务规则: 每个顾客只能拥有一张会员卡

2.2 顾客-地址关系

- 关系类型: 1:N
- 实现方式: 在Address表中添加普通外键
- 业务规则: 一个顾客可以有多个地址

2.3 顾客-电话关系

- 关系类型: 1:N
- 实现方式: 在Phone表中添加普通外键
- 业务规则: 一个顾客可以有多个电话号码

MySQL实现方案

1. 数据库表结构

1.1 顾客表

代码块

```
1 CREATE TABLE Customer (  
2     id_card VARCHAR(18) PRIMARY KEY,  
3     name VARCHAR(50) NOT NULL  
4 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

- 身份证号作为主键, 符合业务唯一性要求
- 姓名设置为NOT NULL, 确保数据完整性

1.2 会员卡表

代码块

```

1 CREATE TABLE MembershipCard (
2     card_number VARCHAR(20) PRIMARY KEY,
3     level VARCHAR(10),
4     points INT DEFAULT 0,
5     customer_id VARCHAR(18) UNIQUE NOT NULL,
6     FOREIGN KEY (customer_id) REFERENCES Customer(id_card)
7         ON DELETE CASCADE
8         ON UPDATE CASCADE
9 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

```

- UNIQUE约束确保1:1关系
- 级联操作保证数据一致性
- 积分默认值为0

1.3 地址表

代码块

```

1 CREATE TABLE Address (
2     address_id INT AUTO_INCREMENT PRIMARY KEY,
3     province VARCHAR(20) NOT NULL,
4     city VARCHAR(20) NOT NULL,
5     district VARCHAR(20),
6     street VARCHAR(100) NOT NULL,
7     customer_id VARCHAR(18) NOT NULL,
8     FOREIGN KEY (customer_id) REFERENCES Customer(id_card)
9         ON DELETE CASCADE
10        ON UPDATE CASCADE,
11     INDEX idx_customer (customer_id)
12 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

```

- 自增主键作为代理键
- 添加索引提高查询效率
- 关键地址信息设置为NOT NULL

1.4 电话表

代码块

```

1 CREATE TABLE Phone (
2     phone_id INT AUTO_INCREMENT PRIMARY KEY,
3     area_code VARCHAR(5) NOT NULL,
4     number VARCHAR(15) NOT NULL,
5     customer_id VARCHAR(18) NOT NULL,

```

```
6      FOREIGN KEY (customer_id) REFERENCES Customer(id_card)
7      ON DELETE CASCADE
8      ON UPDATE CASCADE,
9      INDEX idx_customer (customer_id),
10     UNIQUE uk_phone (area_code, number)
11 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

- 电话号码组合设置唯一约束
- 区号和号码分开存储便于查询
- 添加索引优化查询性能

2. 数据操作示例

2.1 插入数据

代码块

```
1  -- 插入顾客信息
2  INSERT INTO Customer VALUES ('110101199001011234', '张三');
3  -- 插入会员卡
4  INSERT INTO MembershipCard VALUES ('VIP001', 'Gold', 5000,
   '110101199001011234');
5  -- 插入地址
6  INSERT INTO Address (province, city, district, street, customer_id)
7  VALUES ('北京市', '北京市', '海淀区', '中关村大街1号', '110101199001011234');
8  -- 插入电话
9  INSERT INTO Phone (area_code, number, customer_id)
10 VALUES ('010', '12345678', '110101199001011234');
```

2.2 查询示例

代码块

```
1  -- 查询顾客及其会员卡信息
2  SELECT c.name, m.card_number, m.level
3  FROM Customer c
4  JOIN MembershipCard m ON c.id_card = m.customer_id
5  WHERE c.id_card = '110101199001011234';
6  -- 查询顾客所有地址
7  SELECT province, city, district, street
8  FROM Address
9  WHERE customer_id = '110101199001011234';
10 -- 查询顾客所有电话
11 SELECT CONCAT(area_code, '-', number) AS phone
12 FROM Phone
```

```
13 WHERE customer_id = '110101199001011234';
```

实验验证与测试

1. 关系完整性测试

1.1 1:1关系验证

代码块

```
1 -- 尝试为同一顾客添加第二张卡（应失败）
2 INSERT INTO MembershipCard VALUES ('VIP002', 'Silver', 2000,
  '110101199001011234');
3 -- 预期结果: Duplicate entry错误
```

1.2 外键约束验证

代码块

```
1 -- 尝试插入不存在的顾客ID（应失败）
2 INSERT INTO Address VALUES (2, '上海市', '上海市', '浦东新区', '张江高科技园区',
  '99999999999999999999');
3 -- 预期结果: Foreign key constraint fails
```

2. 级联操作测试

2.1 删除顾客

代码块

```
1 DELETE FROM Customer WHERE id_card = '110101199001011234';
2 -- 验证关联的会员卡、地址、电话是否自动删除
```

2.2 更新顾客ID

代码块

```
1 UPDATE Customer SET id_card = '110101199001011235' WHERE id_card =
  '110101199001011234';
2 -- 验证外键是否同步更新
```

实验结论与改进建议

1. 实验成果

- 成功实现了ER图到数据库表的转换
- 正确表达了1:1和1:N关系
- 通过外键约束保证了数据完整性
- 验证了级联操作的有效性

2. 改进建议

2.1 结构优化

- 为地址添加类型字段（家庭/工作/其他）
- 为电话添加类型字段（手机/固话/紧急联系人）
- 会员卡添加有效期字段

2.2 性能优化

代码块

```
1  -- 添加复合索引
2  ALTER TABLE Address ADD INDEX idx_location (province, city, district);
```

2.3 安全增强

- 对身份证号进行加密存储
- 添加数据创建时间戳
- 实现软删除机制

3. 扩展应用

- 可扩展为完整的客户关系管理系统
- 集成会员积分管理功能
- 添加地址地理编码功能

实验心得

通过本次实验，我掌握了以下核心技能：

- 将概念模型(ER图)转换为物理模型的能力
- 关系型数据库中外键约束的使用技巧
- 不同基数关系的实现方法
- 数据库完整性的保障机制

实验过程中，通过反复测试验证了各种约束条件的有效性，特别是级联操作在实际业务中的重要性。这些经验对今后设计更复杂的数据库系统奠定了坚实基础。建议后续可以结合应用开发，实现完整的CRUD操作界面，进一步验证设计方案的实用性。